

Cours de Computer Vision

Fondements, modèles modernes et démarche pratique

Cours

2026

Table des matières

Mode d'emploi	8
1 Images numériques et formulation des tâches	9
1.1 Pourquoi la vision est difficile	9
1.2 Tâches principales	9
1.3 Données et split	10
1.4 Du besoin métier à la tâche d'apprentissage	10
1.5 Résolution, canaux et dynamique	10
1.6 Annotations et qualité de vérité terrain	10
1.7 Échantillonnage et représentativité	11
1.8 Exemple de formulation complète	11
1.9 Questions de contrôle	11
2 Filtres, caractéristiques et convolution	12
2.1 Dimensions	12
2.2 Approche classique et approche apprise	12
2.3 Calcul manuel d'une réponse locale	13
2.4 Contours et dérivées discrètes	13
2.5 Histogrammes et descripteurs	13
2.6 Stride, padding et dilation	13
2.7 Plusieurs canaux et plusieurs filtres	14
2.8 Convolution, équivariance et invariance	14
2.9 Exercices corrigés	14
3 Réseaux de neurones et apprentissage	15
3.1 Losses	15

3.2	Optimisation	15
3.3	Dérivation d'un neurone simple	16
3.4	Initialisation	16
3.5	Optimiseurs en pratique	16
3.6	Régularisation et généralisation	17
3.7	Vanishing et exploding gradients	17
3.8	Protocole de débogage	17
4	Réseaux convolutifs	18
4.1	Architecture type	18
4.2	Variantes efficaces	18
4.3	Pourquoi un réseau dense est insuffisant	19
4.4	Calcul tensoriel d'une couche	19
4.5	Suivi détaillé des formes	19
4.6	Pooling, stride et perte d'information	19
4.7	Batch Normalization	20
4.8	Architecture PyTorch minimale	20
4.9	Visualisation et diagnostic	20
5	Architectures CNN et Transfer Learning	21
5.1	ResNet	21
5.2	Transfer Learning	21
5.3	Lecture historique raisonnée	22
5.4	Bloc résiduel complet	22
5.5	Multi-échelle et efficacité	22
5.6	Protocole complet de Transfer Learning	22
5.7	Quantité de données et distance de domaine	23
5.8	Oubli catastrophique et BatchNorm	23
5.9	Choisir une architecture sous contraintes	23
6	Classification d'images	24
6.1	Déséquilibre	24
6.2	Analyse d'erreurs	24
6.3	Formats de cibles et cohérence de la loss	24

6.4	Seuils et coûts métier	25
6.5	Label smoothing	25
6.6	Top-k et calibration	25
6.7	Analyse d'erreurs structurée	25
6.8	Grad-CAM	26
6.9	Checklist de classification	26
7	Détection d'objets	27
7.1	Intersection over Union	27
7.2	Deux familles	27
7.3	Non-Maximum Suppression	27
7.4	Paramétrisation et conversion des boîtes	28
7.5	Appariement prédiction-vérité terrain	28
7.6	Precision-Recall et Average Precision	28
7.7	Petits, moyens et grands objets	28
7.8	Feature Pyramid Network	28
7.9	Détecteurs à deux étapes	29
7.10	Loss de localisation	29
7.11	Checklist de détection	29
8	YOLO	30
8.1	Sorties et loss	30
8.2	Pipeline	30
8.3	Architecture générale	30
8.4	Anchor-based et anchor-free	31
8.5	Prédiction multi-échelle	31
8.6	Objectness et classes	31
8.7	Assignment des cibles	31
8.8	Letterboxing et retour aux coordonnées originales	31
8.9	Réglage de l'inférence	32
8.10	Entraînement pratique	32
8.11	Déploiement de YOLO	32
9	Segmentation d'images	33

9.1	Encodeur-décodeur	33
9.2	Losses et métriques	33
9.3	Sortie sémantique	33
9.4	U-Net détaillé	34
9.5	Sur-échantillonnage	34
9.6	Déséquilibre entre pixels	34
9.7	Tversky et Focal Tversky	34
9.8	Ignore Index et bordures	34
9.9	Instance et panoptique	35
9.10	Évaluation qualitative	35
9.11	Tiling pour grandes images	35
10	Vision Transformers et modèles vision-langage	36
10.1	DETR et vision-langage	36
10.2	Tokenisation d’une image	36
10.3	Multi-Head Self-Attention	37
10.4	Position et biais inductif	37
10.5	Transformers hiérarchiques	37
10.6	DETR	37
10.7	Apprentissage contrastif vision–langage	38
10.8	Limites des modèles multimodaux	38
11	Données, annotations et augmentation	39
11.1	Augmentations	39
11.2	Concevoir une taxonomie	39
11.3	Mesurer la qualité d’annotation	40
11.4	Active Learning	40
11.5	Augmentations géométriques et photométriques	40
11.6	CutMix et effacement	40
11.7	Contrôle des transformations	40
11.8	Versionner les données	41
12	Métriques et évaluation	42
12.1	Détection	42

12.2	Segmentation et calibration	42
12.3	ROC-AUC et PR-AUC	42
12.4	Balanced Accuracy et MCC	43
12.5	Intervalles et répétitions	43
12.6	Tests par sous-groupes	43
12.7	Seuil et coût attendu	43
12.8	Protocoles reproductibles	43
13	Robustesse, explicabilité et déploiement	45
13.1	Explicabilité	45
13.2	Production	45
13.3	Tests de robustesse	45
13.4	Incertitude et rejet	46
13.5	Compression	46
13.6	Monitoring	46
13.7	Sécurité et confidentialité	46
13.8	Décision finale d'architecture	46
13.9	Choisir une approche face à un problème réel	47
13.10	Protocole final	47
13.11	Conclusion	47
	Exercices transversaux corrigés	48
	Bibliographie sélective	50
A	Travaux pratiques : classification complète	52
A.1	Objectif et structure du projet	52
A.2	Dataset et transformations	52
A.3	Chargement et reproductibilité	53
A.4	Modèle pré-entraîné	53
A.5	Fonctions Train et Validation	53
A.6	Checkpoint et Early Stopping	54
A.7	Questions corrigées	54
B	Travaux pratiques : détection d'objets	56

B.1	Audit des annotations	56
B.2	Transformation cohérente	56
B.3	NMS simplifiée	56
B.4	Analyse d'erreurs	57
B.5	Étude de résolution	57
B.6	Exercice corrigé	57
C	Travaux pratiques : segmentation	58
C.1	Dataset masque-image	58
C.2	Loss combinée	58
C.3	Évaluation par classe et par image	58
C.4	Tiling avec recouvrement	59
C.5	Diagnostic visuel	59
D	Études de cas guidées	60
D.1	Défauts industriels	60
D.2	Imagerie médicale	60
D.3	Caméra embarquée	60
D.4	Catalogue produit	61
E	Exercices supplémentaires corrigés	62
E.1	Réseaux et optimisation	62
E.2	Classification	62
E.3	Détection	62
E.4	Segmentation	62
E.5	Transformers	63
E.6	Question de synthèse	63
F	Glossaire	64
G	Aide-mémoire mathématique	65
H	Fiches de révision par chapitre	66
H.1	Fiche 1 – Formuler le problème	66
H.2	Fiche 2 – Convolution	67

H.3	Fiche 3 – Apprentissage neuronal	68
H.4	Fiche 4 – CNN	69
H.5	Fiche 5 – Architectures et transfert	70
H.6	Fiche 6 – Classification	71
H.7	Fiche 7 – Détection	72
H.8	Fiche 8 – YOLO	73
H.9	Fiche 9 – Segmentation	74
H.10	Fiche 10 – Vision Transformers	75
H.11	Fiche 11 – Données et augmentation	76
H.12	Fiche 12 – Évaluation	77
H.13	Fiche 13 – Robustesse et production	78
I	Aide-mémoire mathématique	79

Mode d'emploi

Ce cours suit une progression allant des pixels à la mise en production. Les encadrés *Réflexe* résument les décisions pratiques ; les encadrés *Attention* signalent les pièges fréquents. Les modèles récents sont présentés comme des prolongements des principes fondamentaux, et non comme un catalogue d'architectures.

Chapitre 1

Images numériques et formulation des tâches

La vision par ordinateur transforme des mesures visuelles en informations utiles. Une image en niveaux de gris est une matrice $X \in \mathbb{R}^{H \times W}$; une image couleur est un tenseur $X \in \mathbb{R}^{H \times W \times C}$, avec généralement $C = 3$ pour RGB. Un batch PyTorch suit souvent la convention $N \times C \times H \times W$.

1.1 Pourquoi la vision est difficile

L'apparence varie avec l'éclairage, le point de vue, l'échelle, l'occultation, le capteur et l'arrière-plan. Il existe une variabilité intra-classe et une similarité inter-classe. Avant toute modélisation, il faut vérifier que l'information nécessaire est réellement visible.

1.2 Tâches principales

Tâche	Sortie	Annotation
Classification	Une classe globale	Label image
Multilabel	Plusieurs catégories	Vecteur binaire
Détection	Classes et boîtes	Bounding boxes
Segmentation sémantique	Classe par pixel	Masque
Segmentation d'instances	Masque par objet	Masques d'instances
Pose	Points caractéristiques	Keypoints
Profondeur	Distance par pixel	Carte de profondeur

Réflexe. Choisir la sortie minimale permettant la décision métier. Si une classe suffit, ne pas payer le coût d'annotation d'une segmentation.

1.3 Données et split

Le dataset doit représenter les conditions de production. Les images corrélées doivent rester dans le même ensemble : patient, vidéo, caméra, site ou objet physique. Le split est construit avant l’augmentation afin d’éviter qu’une variante de la même image apparaisse dans Train et Test.

1.4 Du besoin métier à la tâche d’apprentissage

Le choix de la tâche doit partir de la décision attendue. Pour contrôler une pièce industrielle, une classification suffit si l’on veut seulement accepter ou rejeter la pièce. Une détection devient nécessaire si l’opérateur doit localiser le défaut ; une segmentation est justifiée si sa surface ou sa forme doit être mesurée. Une sortie plus détaillée exige généralement des annotations plus coûteuses.

Réflexe. Poser successivement quatre questions : quelle décision sera prise ? Quelle sortie minimale la permet ? Quelles annotations sont disponibles ? Quel type d’erreur est le plus coûteux ?

1.5 Résolution, canaux et dynamique

Une image codée sur 8 bits utilise souvent des valeurs dans $\{0, \dots, 255\}$. Une conversion courante est

$$x' = \frac{x}{255},$$

mais les images scientifiques ou médicales peuvent employer d’autres unités et profondeurs numériques. Une standardisation par canal s’écrit

$$x'_{cij} = \frac{x_{cij} - \mu_c}{\sigma_c}.$$

La résolution influence le compromis entre détail et coût. Doubler hauteur et largeur multiplie approximativement le nombre de pixels par quatre. Un redimensionnement déformant le ratio peut altérer les formes ; on choisit alors crop, padding ou letterboxing selon la tâche.

1.6 Annotations et qualité de vérité terrain

Une vérité terrain n’est pas nécessairement parfaite. Les annotateurs peuvent diverger sur une frontière, une classe ambiguë ou un objet partiellement visible. Il faut définir un guide précisant taxonomie, cas limites, objets ignorés et règles d’occultation. Pour les tâches sensibles, un sous-ensemble peut être annoté par plusieurs personnes puis arbitré.

Le bruit de label fixe une limite à la performance mesurable. Il convient d’inspecter les exemples à forte loss, sans supposer qu’ils sont tous mal annotés : ils peuvent aussi représenter les cas réellement difficiles et importants.

1.7 Échantillonnage et représentativité

La collecte doit couvrir éclairage, capteur, orientation, arrière-plan, population, saison et conditions dégradées. Un Test issu de la même session que le Train mesure surtout l’interpolation. Un Test externe, provenant d’un autre site ou d’une période ultérieure, renseigne mieux sur la robustesse.

Attention. Un split aléatoire image par image est trompeur si des frames voisines, plusieurs vues du même objet ou plusieurs examens du même patient se retrouvent dans des ensembles différents. La bonne unité du split est le groupe indépendant, pas nécessairement l’image.

1.8 Exemple de formulation complète

Supposons un système devant détecter des fissures. Le besoin métier est de signaler les zones à inspecter. La tâche est une segmentation binaire ; l’entrée est une image RGB, la sortie une probabilité par pixel. Les annotations sont des masques. Le split est réalisé par pièce et par site. Les erreurs critiques sont les fissures manquées : le recall et la sensibilité par taille de fissure deviennent prioritaires. La latence et le taux de fausses alertes restent des contraintes de déploiement.

1.9 Questions de contrôle

1. L’information utile est-elle visible dans l’image ?
2. La sortie choisie correspond-elle à la décision réelle ?
3. Les classes sont-elles définies sans ambiguïté ?
4. Le Test représente-t-il les futures données ?
5. Des groupes corrélés traversent-ils le split ?
6. Les petites classes ou petits objets sont-ils suffisamment représentés ?

Chapitre 2

Filtres, caractéristiques et convolution

Un filtre spatial applique un noyau local à chaque position. Pour un noyau K et une image X , l'opération utilisée dans les bibliothèques de deep learning est généralement une corrélation croisée :

$$Y_{i,j} = \sum_u \sum_v K_{u,v} X_{i+u,j+v}.$$

La sortie est une *feature map*. Les filtres classiques incluent le flou gaussien, la netteté et Sobel. Les gradients horizontaux et verticaux donnent

$$\|\nabla X\| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \text{atan2}(G_y, G_x).$$

2.1 Dimensions

Pour une taille N , un kernel K , un padding P , un stride S et une dilation D :

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - D(K - 1) - 1}{S} \right\rfloor + 1.$$

Le partage des poids et la connectivité locale réduisent fortement le nombre de paramètres. Une couche avec C_{in} canaux, C_{out} filtres et des kernels $K_h \times K_w$ possède

$$K_h K_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$$

paramètres lorsque les biais sont utilisés.

2.2 Approche classique et approche apprise

La vision classique conçoit des descripteurs (HOG, SIFT, histogrammes) puis entraîne un SVM ou un autre modèle. Un CNN apprend simultanément les filtres et la décision finale. Une méthode classique reste pertinente dans un environnement contrôlé lorsque la règle visuelle est simple.

2.3 Calcul manuel d’une réponse locale

Considérons

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}.$$

La réponse locale vaut

$$\langle X, K \rangle = 1 - 0 + 0 - 3 + 2 - 0 = 0.$$

En déplaçant K , on obtient une carte indiquant où cette transition orientée apparaît. Un signe opposé correspond à une transition de direction inverse.

2.4 Contours et dérivées discrètes

Un contour est une variation rapide d’intensité. Les filtres de Sobel

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

approchent les dérivées spatiales. L’amplitude et l’orientation sont

$$M = \sqrt{G_x^2 + G_y^2}, \quad \theta = \text{atan2}(G_y, G_x).$$

Le bruit produit lui aussi de fortes variations; un lissage gaussien avant le calcul des gradients améliore souvent la stabilité, au prix d’une perte de détails.

2.5 Histogrammes et descripteurs

Un histogramme de couleurs résume la distribution chromatique mais ignore largement la position. HOG divise l’image en cellules, accumule les orientations de gradient puis normalise des blocs voisins. SIFT détecte des points d’intérêt à plusieurs échelles et décrit leur voisinage; ORB fournit une alternative binaire plus légère. Ces représentations restent utiles pour la mise en correspondance, la géométrie multi-vues et les environnements disposant de peu de données.

2.6 Stride, padding et dilation

Le stride contrôle l’espace des positions calculées. Le padding préserve les bords ou la résolution. La dilation espace les coefficients du kernel et donne une taille effective

$$K_{eff} = D(K - 1) + 1.$$

Ainsi, un kernel 3×3 avec $D = 2$ couvre une zone 5×5 sans ajouter de poids. Une forte dilation peut toutefois créer un échantillonnage en grille qui manque des structures locales.

2.7 Plusieurs canaux et plusieurs filtres

Pour une entrée RGB, chaque filtre possède une profondeur de trois canaux. Les contributions sont sommées avant l'ajout du biais. Avec C_{out} filtres, la sortie possède C_{out} feature maps. Le filtre ne détecte donc pas seulement une couleur isolée : il peut apprendre une combinaison spatiale et chromatique.

2.8 Convolution, équivariance et invariance

Si T_Δ traduit une image, une convolution idéale vérifie

$$f(T_\Delta X) = T_\Delta f(X).$$

Elle est équivariante, non automatiquement invariante. Pooling, agrégation globale, profondeur et augmentation construisent une invariance partielle. Stride, padding et bords rendent cette propriété imparfaite en pratique.

2.9 Exercices corrigés

Pour une entrée 32×32 , $K = 3$, $P = 1$ et $S = 2$:

$$N_{out} = \left\lfloor \frac{32 + 2 - 3}{2} \right\rfloor + 1 = 16.$$

Pour $C_{in} = 16$, $C_{out} = 32$ et $K = 3$, le nombre de paramètres avec biais est

$$3^2 \times 16 \times 32 + 32 = 4640.$$

Deux convolutions 3×3 de stride 1 donnent un champ réceptif 5×5 et introduisent deux non-linéarités, ce qui explique leur intérêt par rapport à un unique grand filtre.

Chapitre 3

Réseaux de neurones et apprentissage

Un neurone calcule

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = f(z).$$

Sans activation non linéaire, une composition de couches affines reste affine. ReLU, $\max(0, z)$, est une activation classique ; sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

sert notamment aux sorties binaires et multilabel. Softmax transforme K logits en probabilités exclusives :

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

3.1 Losses

Pour le binaire :

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Pour le multiclasse :

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_i \log p_{i, y_i}.$$

Les implémentations stables reçoivent directement les logits : ne pas appliquer sigmoid avant `BCEWithLogitsLoss`, ni softmax avant `CrossEntropyLoss`.

3.2 Optimisation

La rétropropagation calcule $\nabla_{\theta} \mathcal{L}$ par la règle de chaîne. La descente de gradient met à jour

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).$$

SGD avec Momentum et AdamW sont des optimiseurs usuels. Le learning rate est central. Une epoch parcourt le Train ; une itération traite un mini-batch. Suivre séparément les courbes Train et Validation permet de détecter underfitting et overfitting.

Réflexe. Avant un entraînement long, vérifier que le modèle peut mémoriser un très petit batch. Sinon, chercher une erreur de données, dimensions, loss ou gradients.

3.3 Dérivation d'un neurone simple

Pour $z = \mathbf{w}^\top \mathbf{x} + b$, $a = \sigma(z)$ et

$$\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)],$$

la combinaison sigmoid-BCE se simplifie en

$$\frac{\partial \mathcal{L}}{\partial z} = a - y.$$

Par conséquent,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (a - y)\mathbf{x}, \quad \frac{\partial \mathcal{L}}{\partial b} = a - y.$$

Cette formule montre que le signe et l'amplitude de l'erreur déterminent la correction, pondérée par chaque entrée.

3.4 Initialisation

Des poids tous identiques empêchent les neurones d'apprendre des fonctions différentes. Xavier vise une variance proche de $2/(d_{in} + d_{out})$; He utilise approximativement $2/d_{in}$ avec ReLU. Une mauvaise échelle initiale favorise activations ou gradients qui disparaissent ou explosent.

3.5 Optimiseurs en pratique

SGD avec Momentum accumule une direction

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \mathbf{v}_t.$$

Adam adapte le pas à partir des moments du gradient ; AdamW découple correctement le weight decay. Aucun optimiseur ne dispense de régler le learning rate. Un scheduler peut employer warm-up, décroissance par paliers ou cosine annealing.

3.6 Régularisation et généralisation

La pénalisation L_2 ajoute

$$\mathcal{L}_{tot} = \mathcal{L}_{data} + \lambda \|\theta\|_2^2.$$

Dropout annule aléatoirement des activations pendant le Train. Early Stopping conserve l'époque minimisant la loss de Validation. La data augmentation agit directement sur la distribution des entrées. Ces méthodes répondent à des mécanismes différents et peuvent être combinées.

3.7 Vanishing et exploding gradients

La règle de chaîne multiplie de nombreuses Jacobiennes. Si leurs normes sont souvent inférieures à un, le gradient disparaît ; si elles sont trop grandes, il explose. ReLU, initialisation adaptée, normalisation et connexions résiduelles facilitent l'optimisation. Le gradient clipping impose par exemple

$$\mathbf{g} \leftarrow \mathbf{g} \min \left(1, \frac{c}{\|\mathbf{g}\|} \right).$$

3.8 Protocole de débogage

1. Vérifier quelques entrées et labels visuellement.
2. Contrôler formes, types et plages numériques.
3. Exécuter un forward pass et vérifier l'absence de NaN.
4. Confirmer la compatibilité logits-labels-loss.
5. Vérifier que les gradients sont non nuls et finis.
6. Mémoriser un petit batch.
7. Lancer ensuite un entraînement court et observer Train/Validation.

Attention. Une accuracy au hasard peut provenir d'un bug silencieux : ordre des classes différent, labels décalés, paramètres absents de l'optimiseur ou prétraitement incohérent. Augmenter le modèle avant ces vérifications complique seulement le diagnostic.

Chapitre 4

Réseaux convolutifs

Une couche reçoit $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ et produit $Y \in \mathbb{R}^{N \times C_{out} \times H' \times W'}$. Les premières couches apprennent souvent bords et couleurs ; les couches profondes combinent textures, parties et concepts.

4.1 Architecture type

```
Image -> [Conv -> BatchNorm -> ReLU] x n
        -> reduction spatiale
        -> Global Average Pooling
        -> couche lineaire -> logits
```

Le champ réceptif croît avec la profondeur. Pour un kernel K_l , un stride S_l , un saut j_l et un champ r_l :

$$j_l = j_{l-1} S_l, \quad r_l = r_{l-1} + (K_l - 1) j_{l-1}.$$

Le pooling réduit la résolution ; le Global Average Pooling calcule

$$g_c = \frac{1}{HW} \sum_i \sum_j A_{c,i,j}.$$

4.2 Variantes efficaces

Une convolution 1×1 combine les canaux. Une convolution dilatée possède une taille effective $K_{eff} = D(K - 1) + 1$. Une convolution depthwise separable réduit le coût de $K^2 C_{in} C_{out}$ à environ $K^2 C_{in} + C_{in} C_{out}$ par position.

4.3 Pourquoi un réseau dense est insuffisant

Une image RGB 224×224 contient 150 528 valeurs. Une couche dense vers 1 000 neurones demanderait

$$150\,528 \times 1\,000 + 1\,000 = 150\,529\,000$$

paramètres. Elle ignore en outre que le même motif peut apparaître partout. La convolution remplace cette connexion globale par un voisinage local et des poids partagés.

4.4 Calcul tensoriel d'une couche

Pour $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, les poids ont la forme

$$W \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}.$$

Une activation est

$$z_{cij} = b_c + \sum_{d=1}^{C_{in}} \sum_u \sum_v W_{cd uv} X_{d, iS_h+u-P_h, jS_w+v-P_w}.$$

Chaque filtre produit un canal de sortie. Pour 64 filtres appliqués à une image RGB, la profondeur de sortie est 64, pas 3.

4.5 Suivi détaillé des formes

Considérons le réseau suivant :

3x32x32	-> Conv(3,32,3,pad=1)	-> 32x32x32
	-> MaxPool(2)	-> 32x16x16
	-> Conv(32,64,3,pad=1)	-> 64x16x16
	-> MaxPool(2)	-> 64x8x8
	-> GlobalAvgPool	-> 64
	-> Linear(64,K)	-> K logits

La première convolution contient $3^2 \times 3 \times 32 + 32 = 896$ paramètres ; la seconde $3^2 \times 32 \times 64 + 64 = 18\,496$. Une grande dimension spatiale augmente le calcul et la mémoire d'activation, mais pas le nombre de poids du kernel.

4.6 Pooling, stride et perte d'information

Max pooling conserve le maximum d'une région ; average pooling en conserve la moyenne. Une convolution de stride 2 apprend simultanément filtrage et réduction. Toute réduction spatiale détruit de l'information fine : acceptable pour une classification globale, elle doit être compensée dans une segmentation par skip connections ou décodage.

4.7 Batch Normalization

Pour un mini-batch, une activation est transformée par

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta.$$

Les statistiques du batch sont utilisées en mode Train ; des moyennes accumulées servent en mode Eval. Avec de très petits batches, GroupNorm peut être plus stable. BatchNorm facilite l'optimisation mais ne remplace ni un bon learning rate ni un bon split.

4.8 Architecture PyTorch minimale

```
class SimpleCNN(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2))
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.head = nn.Linear(64, n_classes)
    def forward(self, x):
        x = self.pool(self.features(x))
        return self.head(torch.flatten(x, 1))
```

Le forward retourne des logits. Pour N images et K classes, la forme finale doit être $N \times K$.

4.9 Visualisation et diagnostic

Les kernels de première couche sont parfois lisibles comme détecteurs de couleur ou d'orientation. Les feature maps indiquent les réponses locales, mais leur interprétation devient difficile en profondeur. Avant l'entraînement, tester un batch artificiel, imprimer chaque forme, vérifier les valeurs finies et compter les paramètres. Une erreur après Flatten provient souvent d'une hypothèse fautive sur H et W ; Adaptive Average Pooling évite une partie de cette fragilité.

Réflexe. Pour chaque bloc, écrire explicitement $C \times H \times W$, le champ réceptif et le nombre de paramètres. La profondeur augmente généralement la sémantique et le champ réceptif, tandis que la résolution diminue.

Chapitre 5

Architectures CNN et Transfer Learning

Les architectures doivent être retenues par leur idée principale : LeNet établit le pipeline ; AlexNet démontre l'entraînement profond sur GPU ; VGG empile des kernels 3×3 ; Inception traite plusieurs échelles ; MobileNet emploie des convolutions séparables ; EfficientNet équilibre profondeur, largeur et résolution ; ConvNeXt modernise le CNN.

5.1 ResNet

Un bloc résiduel apprend

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}.$$

Le gradient contient un chemin identité :

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + I.$$

Si les dimensions changent, une projection donne $\mathbf{y} = F(\mathbf{x}) + W_s \mathbf{x}$. DenseNet concatène plutôt les sorties antérieures.

5.2 Transfer Learning

Un backbone pré-entraîné fournit une représentation $\mathbf{h}$; une nouvelle tête produit $\mathbf{z} = W\mathbf{h} + \mathbf{b}$. Stratégie recommandée : remplacer la tête, geler le backbone, entraîner la tête, puis dégeler progressivement les derniers blocs avec $\eta_{backbone} < \eta_{head}$. Le prétraitement doit correspondre aux poids pré-entraînés.

Attention. Geler les gradients n'est pas identique au mode évaluation : BatchNorm et Dropout doivent être contrôlés explicitement.

5.3 Lecture historique raisonnée

LeNet établit convolution–pooling–classification. AlexNet combine GPU, ReLU, augmentation et Dropout. VGG régularise la conception avec des kernels 3×3 . Inception traite plusieurs échelles en parallèle. Ces architectures sont importantes pour leurs idées, pas nécessairement comme choix de production actuels.

5.4 Bloc résiduel complet

Dans un Basic Block post-activation, la branche principale suit typiquement

<code>Conv 3x3 -> BatchNorm -> ReLU -> Conv 3x3 -> BatchNorm</code>

puis l'on additionne le raccourci et applique ReLU. Si les formes diffèrent, une convolution 1×1 avec stride adapte le raccourci :

$$\mathbf{y} = \text{ReLU}(F(\mathbf{x}) + W_s \mathbf{x}).$$

ResNet-18 et 34 utilisent surtout le Basic Block ; les variantes 50 et plus utilisent un bottleneck $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$.

5.5 Multi-échelle et efficacité

Inception concatène plusieurs branches :

$$Y = \text{Concat}(f_{1 \times 1}(X), f_{3 \times 3}(X), f_{5 \times 5}(X), f_{\text{pool}}(X)).$$

MobileNet sépare filtrage spatial et combinaison des canaux. EfficientNet fait varier profondeur, largeur et résolution selon

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi,$$

avec une contrainte de coût approximative. ConvNeXt réintroduit dans un CNN des choix modernes : grands kernels depthwise, normalisation et organisation en stages.

5.6 Protocole complet de Transfer Learning

1. Charger les poids et leur prétraitement associé.
2. Remplacer la tête avec le bon nombre de sorties.
3. Geler le backbone et entraîner la tête.
4. Mesurer la baseline sur Validation.
5. Dégeler le dernier stage et utiliser un faible learning rate.
6. Dégeler davantage seulement si la validation progresse.

7. Conserver le meilleur checkpoint et évaluer une fois sur Test.

Une règle courante est

$$\eta_{early} < \eta_{late} < \eta_{head}.$$

Les premières couches sont générales et modifiées prudemment ; la tête aléatoire doit apprendre plus vite.

5.7 Quantité de données et distance de domaine

Avec peu de données et un domaine proche, feature extraction est un excellent départ. Avec davantage de données, fine-tuner les derniers stages. Un domaine éloigné (radiographie versus photographie naturelle) peut nécessiter un pré-entraînement spécialisé ou un dégel plus large. Ce choix reste empirique : comparer sous un protocole identique.

5.8 Oubli catastrophique et BatchNorm

Un learning rate trop élevé peut détruire rapidement les représentations pré-entraînées. Le dégel progressif, Early Stopping et des learning rates discriminatifs limitent ce risque. Les statistiques BatchNorm peuvent évoluer même si les poids sont gelés ; décider explicitement si elles doivent rester fixes ou s'adapter au domaine cible.

5.9 Choisir une architecture sous contraintes

Comparer non seulement accuracy mais paramètres, FLOPs, mémoire maximale, latence par image et débit. ResNet-18 constitue une baseline simple ; MobileNet vise l'embarqué ; des variantes EfficientNet ou ConvNeXt offrent souvent un meilleur compromis précision-coût. La mesure doit être faite sur le matériel et avec le batch de production.

Réflexe. Commencer par un backbone pré-entraîné simple. N'augmenter taille, résolution ou profondeur qu'après une analyse d'erreurs montrant que la capacité est réellement limitante.

Chapitre 6

Classification d'images

En binaire, utiliser un logit et BCE. En multiclass exclusif, utiliser K logits et Cross-Entropy. En multilabel, utiliser K sigmoids indépendantes et BCE. Pour le multiclass :

$$\hat{y} = \arg \max_k z_k.$$

6.1 Déséquilibre

Une Cross-Entropy pondérée s'écrit

$$\mathcal{L} = -\frac{1}{N} \sum_i w_{y_i} \log p_{i,y_i}.$$

La Focal Loss concentre l'apprentissage sur les exemples difficiles :

$$\mathcal{L}_{focal} = -\alpha_t (1 - p_t)^\gamma \log p_t.$$

Évaluer avec matrice de confusion, precision, recall et Macro F1, pas seulement accuracy.

6.2 Analyse d'erreurs

Inspecter les erreurs fortement confiantes, les classes rares et les sous-groupes. Rechercher les raccourcis visuels : arrière-plan, filigrane ou capteur corrélé au label. Grad-CAM peut localiser les régions influentes, sans constituer une preuve causale.

6.3 Formats de cibles et cohérence de la loss

En binaire, un modèle peut produire un logit $z \in \mathbb{R}$ et employer BCEWithLogitsLoss. En multiclass exclusif, il produit $Z \in \mathbb{R}^{N \times K}$ et des indices $y_i \in \{0, \dots, K-1\}$. En multilabel, la cible est $Y \in \{0, 1\}^{N \times K}$ et chaque classe possède une sigmoid indépendante. Une grande part des erreurs d'entraînement provient d'une incohérence entre ces trois cas.

6.4 Seuils et coûts métier

En binaire ou multilabel,

$$\hat{y}_k = \mathbb{I}[p_k \geq \tau_k].$$

Abaisser τ_k augmente généralement le recall et les faux positifs ; l'augmenter favorise la precision et les faux négatifs. Les seuils sont choisis sur Validation, selon une métrique ou un coût métier, jamais sur Test.

6.5 Label smoothing

Pour K classes et un lissage ε , une cible peut devenir

$$y_k^{LS} = \begin{cases} 1 - \varepsilon + \varepsilon/K & k = y, \\ \varepsilon/K & k \neq y. \end{cases}$$

Cette régularisation limite parfois la surconfiance, mais un lissage excessif réduit la séparation et peut affecter la calibration.

6.6 Top-k et calibration

La Top- k accuracy vaut

$$\frac{1}{N} \sum_i \mathbb{I}[y_i \in \text{TopK}(z_i)].$$

Elle est pertinente lorsque plusieurs suggestions sont acceptables. Un score softmax élevé n'est pas automatiquement une probabilité correcte. L'ECE compare confiance et fréquence de succès :

$$ECE = \sum_m \frac{|B_m|}{N} |acc(B_m) - conf(B_m)|.$$

Temperature scaling peut ajuster les logits par z/T sur Validation sans changer leur ordre.

6.7 Analyse d'erreurs structurée

Créer quatre groupes : correct et confiant ; correct mais incertain ; faux et incertain ; faux et confiant. Le dernier groupe révèle souvent labels erronés, raccourcis ou shift de domaine. Croiser les erreurs avec classe, site, capteur, luminosité, résolution et taille de l'objet.

6.8 Grad-CAM

Pour la classe c et les feature maps A^k :

$$\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial z_c}{\partial A_{ij}^k}, \quad L^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right).$$

La carte sert à formuler des hypothèses sur les régions utilisées. Elle ne démontre ni causalité ni absence d'utilisation d'autres informations.

6.9 Checklist de classification

1. Vérifier taxonomie, labels et ordre des classes.
2. Construire un split par groupe avant toute augmentation.
3. Établir classe majoritaire et backbone gelé comme baselines.
4. Suivre loss, accuracy, Macro F1 et matrice de confusion.
5. Régler pondérations et seuils sur Validation.
6. Inspecter erreurs fortes et sous-groupes.
7. Tester calibration, rejet et robustesse externe.
8. Sauvegarder poids, prétraitement, classes et seuils.

Attention. Une excellente accuracy peut provenir d'un arrière-plan, d'un texte ou d'un dispositif corrélé au label. Tester des fonds nouveaux, masquer des zones suspectes et utiliser un Test provenant d'un autre contexte.

Chapitre 7

Détection d'objets

La détection prédit pour chaque objet une classe, une boîte et un score. Une boîte peut être représentée par $(x_{min}, y_{min}, x_{max}, y_{max})$ ou (x_c, y_c, w, h) .

7.1 Intersection over Union

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Une détection est associée à un objet réel selon un seuil d'IoU, puis classée TP, FP ou FN. Les objets manqués sont des FN ; doublons et mauvaises localisations deviennent souvent des FP.

7.2 Deux familles

Les détecteurs à deux étapes (Faster R-CNN) proposent des régions puis les classent : bonne précision, coût supérieur. Les détecteurs à une étape prédisent directement sur des feature maps : meilleure vitesse. Les Feature Pyramid Networks combinent plusieurs résolutions pour traiter les objets de tailles différentes.

7.3 Non-Maximum Suppression

NMS trie les boîtes par score, conserve la meilleure et supprime les boîtes très chevauchantes de même classe. Un seuil trop faible supprime des objets proches ; trop élevé conserve des doublons.

7.4 Paramétrisation et conversion des boîtes

Une boîte (x_c, y_c, w, h) se convertit en coins par

$$x_{min} = x_c - \frac{w}{2}, \quad x_{max} = x_c + \frac{w}{2}, \quad y_{min} = y_c - \frac{h}{2}, \quad y_{max} = y_c + \frac{h}{2}.$$

Les coordonnées peuvent être exprimées en pixels ou normalisées dans $[0, 1]$. Toute augmentation géométrique doit transformer l'image et ses boîtes avec la même opération, puis borner ou supprimer les boîtes invalides.

7.5 Appariement prédiction-vérité terrain

Pour une classe et un seuil d'IoU, les prédictions sont triées par confiance. Une prédiction devient TP si elle correspond à un objet réel encore non associé ; sinon elle devient FP. Un objet réel jamais associé est FN. Cette règle empêche plusieurs boîtes de revendiquer le même objet.

7.6 Precision-Recall et Average Precision

En faisant varier le seuil de confiance, on obtient

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}.$$

L'Average Precision résume l'aire sous la courbe precision-recall. La mAP moyenne l'AP sur les classes et parfois sur plusieurs seuils d'IoU. Il faut toujours préciser le protocole : par exemple AP à IoU 0.50, ou moyenne sur 0.50 : 0.05 : 0.95.

7.7 Petits, moyens et grands objets

Une mAP globale peut masquer l'échec sur les petits objets. Ceux-ci occupent peu de pixels, disparaissent lors des réductions spatiales et sont plus sensibles aux erreurs de coordonnées. Solutions possibles : résolution supérieure, pyramides de features, crops, tiling et collecte ciblée.

7.8 Feature Pyramid Network

Une FPN combine une voie descendante sémantique et des connexions latérales. Les features profondes sont sur-échantillonnées puis fusionnées avec des cartes plus fines :

$$P_l = \text{Conv}_{3 \times 3} (C_l + \text{Up}(P_{l+1})).$$

Les différents niveaux servent à détecter des objets de tailles différentes.

7.9 Détecteurs à deux étapes

Faster R-CNN utilise un Region Proposal Network, puis extrait et classe les régions proposées. La régression de boîte affine leur géométrie. Cette famille privilégie souvent précision et flexibilité, avec une latence supérieure aux détecteurs one-stage.

7.10 Loss de localisation

Une loss L_1 lisse peut pénaliser les différences de coordonnées. Les losses IoU, GIoU, DIoU ou CIoU optimisent directement le recouvrement et certains aspects géométriques. Une forme de GIoU est

$$GIoU = IoU - \frac{|C \setminus (A \cup B)|}{|C|},$$

où C est la plus petite boîte contenant A et B .

7.11 Checklist de détection

Inspecter séparément : objets manqués, mauvaise classe, localisation insuffisante, doublons, faux objets et erreurs par taille. Vérifier les conventions de coordonnées, les transformations d'annotations, le seuil de confiance et le seuil NMS.

Attention. Une détection visuellement plausible peut être comptée fausse si son IoU est sous le seuil. Inversement, une bonne mAP n'assure pas que la latence ou le taux de fausses alertes soit acceptable en production.

Chapitre 8

YOLO

YOLO est une famille de détecteurs one-stage orientés vers un bon compromis vitesse-précision. Un backbone extrait les features, un neck fusionne les échelles, puis une head prédit classes, objectness et géométrie.

8.1 Sorties et loss

Une loss schématique est

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj} + \lambda_{cls}\mathcal{L}_{cls}.$$

Les versions modernes peuvent être anchor-based ou anchor-free. Il faut retenir les principes plutôt qu'un numéro de version : prédiction multi-échelle, attribution des cibles, loss de boîte et post-traitement.

8.2 Pipeline

```
Image -> resize/letterbox -> reseau -> boites brutes  
      -> seuil de confiance -> NMS -> detections finales
```

Pour déployer, mesurer mAP, recall, latence, débit, mémoire et performance par taille d'objet. Une résolution plus grande aide les petits objets mais augmente le coût.

8.3 Architecture générale

Le backbone extrait des features ; le neck fusionne plusieurs échelles ; la head produit les prédictions. Une sortie locale peut contenir géométrie, objectness et scores de classes. Les versions modernes diffèrent dans leurs blocs, leur stratégie d'assignation et leur post-traitement, mais ce schéma reste stable.

8.4 Anchor-based et anchor-free

Un détecteur anchor-based part de boîtes de référence de tailles et ratios prédéfinis, puis prédit des corrections. Il faut adapter les anchors à la distribution des objets. Un détecteur anchor-free prédit plus directement centres, distances aux bords ou coins. Anchor-free simplifie certains réglages sans supprimer le problème d’attribution des cibles.

8.5 Prédiction multi-échelle

Une tête haute résolution traite les petits objets ; une tête basse résolution dispose de davantage de contexte pour les grands. Pour une image redimensionnée, vérifier que les objets les plus petits restent représentés par suffisamment de pixels.

8.6 Objectness et classes

L’objectness estime la présence d’un objet. Un score final peut combiner

$$s_k = P(\text{objet})P(k \mid \text{objet}).$$

Selon l’implémentation, classes et objectness sont formulées différemment ; il faut lire la définition exacte avant d’interpréter ou modifier les seuils.

8.7 Assignment des cibles

Pendant l’entraînement, chaque objet réel doit être associé à une ou plusieurs positions de prédiction. Une assignation trop restrictive réduit les positifs ; trop large crée des conflits. Les méthodes modernes utilisent souvent IoU, proximité du centre et coût combiné pour sélectionner dynamiquement les prédictions responsables.

8.8 Letterboxing et retour aux coordonnées originales

Le letterboxing conserve le ratio puis ajoute des marges. Si le facteur d’échelle est r et les marges (p_x, p_y) , le retour s’écrit

$$x_{orig} = \frac{x_{net} - p_x}{r}, \quad y_{orig} = \frac{y_{net} - p_y}{r}.$$

Oublier cette inversion décale toutes les boîtes affichées.

8.9 Réglage de l'inférence

Le seuil de confiance contrôle le nombre de candidats ; le seuil NMS contrôle la suppression des recouvrements. Ils doivent être réglés sur Validation selon le coût des alertes et des objets manqués. Pour des objets très proches, Soft-NMS ou d'autres stratégies peuvent éviter des suppressions excessives.

8.10 Entraînement pratique

Commencer par des poids pré-entraînés, vérifier les annotations visuellement, entraîner à résolution raisonnable puis analyser les erreurs. Les augmentations mosaïque, MixUp et transformations géométriques peuvent aider mais doivent rester plausibles. Sur un petit dataset, surveiller fortement l'overfitting et la fuite entre scènes.

8.11 Déploiement de YOLO

Mesurer la chaîne complète : décodage, prétraitement, réseau, NMS et transfert des résultats. Une latence GPU seule ne représente pas toujours l'application. Le batch 1 est pertinent pour le temps réel ; de grands batches mesurent plutôt le débit.

Réflexe. Pour améliorer YOLO, classer d'abord l'erreur : objet trop petit, classe confondue, boîte imprécise, score faible, doublon ou domaine absent. Chaque cause appelle une correction différente.

Chapitre 9

Segmentation d'images

La segmentation sémantique prédit une classe par pixel ; la segmentation d'instances distingue les objets ; la segmentation panoptique combine *stuff* et *things*.

9.1 Encodeur-décodeur

L'encodeur réduit la résolution et construit le contexte. Le décodeur reconstruit une carte dense. U-Net transmet des features haute résolution par skip connections :

$$F_{dec}^{(l)} = \text{Concat}(\text{Up}(F_{dec}^{(l+1)}), F_{enc}^{(l)}).$$

DeepLab emploie notamment des convolutions dilatées et un contexte multi-échelle. Mask R-CNN ajoute une tête de masque à un détecteur d'instances.

9.2 Losses et métriques

Pour une classe :

$$Dice = \frac{2|P \cap G|}{|P| + |G|}, \quad IoU = \frac{|P \cap G|}{|P \cup G|}.$$

Une version lissée de la Dice Loss est

$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_i p_i g_i + \varepsilon}{\sum_i p_i + \sum_i g_i + \varepsilon}.$$

La combinaison Cross-Entropy + Dice est fréquente lorsque le fond domine.

9.3 Sortie sémantique

Pour K classes, le réseau produit

$$Z \in \mathbb{R}^{N \times K \times H \times W}.$$

La prédiction pixel par pixel est $\hat{y}_{ij} = \arg \max_k Z_{kij}$. En binaire, un seul logit par pixel et BCEWithLogitsLoss suffisent.

9.4 U-Net détaillé

L'encodeur construit le contexte et réduit la résolution. Le décodeur augmente la résolution. Les skip connections concatènent les détails spatiaux de l'encodeur avec les features sémantiques du décodeur. Les formes doivent être compatibles ; crop ou interpolation peuvent être nécessaires selon padding et dimensions.

9.5 Sur-échantillonnage

L'interpolation bilinéaire est fixe et stable. La convolution transposée est apprise mais peut produire des artefacts en damier si kernel et stride interagissent mal. Une stratégie robuste est interpolation suivie d'une convolution ordinaire.

9.6 Déséquilibre entre pixels

Le fond peut représenter presque toute l'image. La pixel accuracy devient alors trompeuse. Pondération par classe, focal loss, Dice ou Tversky permettent de concentrer l'apprentissage sur les régions rares.

9.7 Tversky et Focal Tversky

$$Tversky = \frac{TP}{TP + \alpha FP + \beta FN}.$$

Choisir $\beta > \alpha$ pénalise davantage les faux négatifs. Une Focal Tversky Loss peut être écrite

$$\mathcal{L} = (1 - Tversky)^\gamma.$$

Elle est utile lorsque les objets segmentés sont petits, mais ses hyperparamètres doivent être validés.

9.8 Ignore Index et bordures

Certains pixels peuvent être non annotés ou ambigus. Un ignore index les exclut de la loss et des métriques. Les frontières sont souvent plus incertaines que l'intérieur ; une métrique de contour ou une tolérance spatiale peut compléter IoU et Dice.

9.9 Instance et panoptique

La segmentation d'instances associe classe, boîte et masque à chaque objet. La panoptique fusionne les instances de *things* avec les régions de *stuff*. La Panoptic Quality combine reconnaissance et qualité des masques :

$$PQ = \underbrace{\frac{\sum_{(p,g) \in TP} IoU(p,g)}{|TP|}}_{SQ} \times \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{RQ}.$$

9.10 Évaluation qualitative

Superposer prédiction et vérité terrain. Inspecter petits objets, trous, débordements, confusion de classes et limites. Calculer les métriques par classe et par image : une moyenne globale masque les images totalement ratées.

9.11 Tiling pour grandes images

Pour une image très grande, découper en patches chevauchants puis fusionner les probabilités. Le chevauchement limite les artefacts de bord. Le split reste réalisé au niveau de l'image ou du patient avant la découpe, sinon des patches voisins contaminent Test.

Réflexe. Si la classification globale est correcte mais les contours sont mauvais, augmenter simplement la profondeur n'est pas la première réponse. Vérifier résolution, réduction spatiale, skip connections, qualité des masques et loss de région ou de frontière.

Chapitre 10

Vision Transformers et modèles vision-langage

Une image est découpée en patches. Chaque patch aplati est projeté :

$$\mathbf{e}_i = W_p \mathbf{x}_i + \mathbf{p}_i,$$

où $\mathbf{p}_i$ encode la position. L'attention est

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Le coût quadratique en nombre de tokens motive fenêtres locales et architectures hiérarchiques.

10.1 DETR et vision-langage

DETR formule la détection comme une prédiction d'ensemble avec appariement biparti. CLIP rapproche représentations image et texte dans un espace commun, permettant notamment du zero-shot. Les modèles multimodaux peuvent décrire et interroger une image, mais restent sensibles aux hallucinations, biais et détails fins.

Complément. Les modèles de fondation ne suppriment pas le besoin d'un Test représentatif, de métriques métier et d'une validation humaine pour les usages critiques.

10.2 Tokenisation d'une image

Pour une image $H \times W$ découpée en patches $P \times P$, le nombre de tokens est

$$N = \frac{HW}{P^2}.$$

Chaque patch RGB aplati possède $3P^2$ valeurs. Une projection linéaire le transforme en dimension d . Un token de classification peut être ajouté ; sinon on agrège les tokens par moyenne.

10.3 Multi-Head Self-Attention

Pour chaque tête h :

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V,$$

$$head_h = \text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{d_h}} \right) V_h.$$

Les têtes sont concaténées puis projetées. Le bloc ajoute résidus, LayerNorm et MLP. L'attention globale permet des interactions lointaines, mais coûte $O(N^2)$ en nombre de tokens.

10.4 Position et biais inductif

Contrairement à un CNN, un Transformer ne possède pas naturellement localité et partage spatial de la même manière. Les embeddings positionnels indiquent l'organisation des patches. Les CNN sont souvent efficaces avec moins de données grâce à leurs biais inductifs ; les Transformers bénéficient fortement du pré-entraînement à grande échelle.

10.5 Transformers hiérarchiques

Des architectures comme Swin limitent l'attention à des fenêtres puis déplacent celles-ci entre blocs. Elles construisent plusieurs résolutions, utiles en détection et segmentation, tout en réduisant le coût par rapport à l'attention globale.

10.6 DETR

DETR prédit un ensemble fixe de détections. Un appariement biparti associe prédictions et objets réels en minimisant un coût combinant classe et boîte :

$$\hat{\sigma} = \arg \min_{\sigma} \sum_i \mathcal{C}(y_i, \hat{y}_{\sigma(i)}).$$

Cette formulation réduit le besoin d'anchors et de NMS classique. Les variantes récentes améliorent convergence, multi-échelle et petits objets.

10.7 Apprentissage contrastif vision–langage

Pour des embeddings image u_i et texte v_j normalisés, une similarité est

$$s_{ij} = \frac{u_i^\top v_j}{\tau}.$$

La loss favorise les paires correspondantes et repousse les autres. En zero-shot, on encode des prompts textuels de classes et l’on choisit celui dont l’embedding est le plus proche de l’image.

10.8 Limites des modèles multimodaux

Ils peuvent halluciner, manquer des détails spatiaux, reproduire des biais ou échouer sur des domaines spécialisés. Les prompts influencent les résultats. Un système critique doit conserver un protocole de données, des métriques et une possibilité de rejet ou de validation humaine.

Réflexe. Choisir un ViT ou un modèle fondation pour ses représentations et son échelle de pré-entraînement, pas uniquement parce qu’il est récent. Comparer à un CNN pré-entraîné sous le même protocole et le même budget.

Chapitre 11

Données, annotations et augmentation

Une taxonomie claire définit classes, cas limites, occultations et objets ignorés. Mesurer les désaccords entre annotateurs et auditer les labels. Les splits doivent refléter les groupes et la chronologie de production.

11.1 Augmentations

Une transformation T est valide si elle conserve la cible :

$$y(T(x)) = y(x).$$

Transformations courantes : crop, flip, rotation modérée, color jitter, bruit, blur, Cutout, MixUp et CutMix. En détection et segmentation, transformer simultanément image, boîtes, masques et keypoints.

MixUp construit

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j.$$

Une augmentation trop forte peut détruire le signal ou créer des images irréalistes. Ne jamais augmenter Validation et Test avec des transformations aléatoires ordinaires.

11.2 Concevoir une taxonomie

Chaque classe doit posséder une définition positive, des exclusions et des exemples limites. Une classe “autre” trop hétérogène est difficile à apprendre. Une hiérarchie peut être utile, par exemple véhicule $\rightarrow$ voiture/camion/bus, mais la sortie et l’évaluation doivent refléter cette structure.

11.3 Mesurer la qualité d’annotation

Pour deux annotateurs en classification, Cohen κ corrige l’accord attendu au hasard :

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

Pour boîtes et masques, on peut mesurer IoU ou Dice entre annotateurs. Un désaccord élevé peut révéler un guide imprécis ou une tâche intrinsèquement ambiguë.

11.4 Active Learning

Lorsque l’annotation est coûteuse, on peut sélectionner en priorité les exemples informatifs : forte incertitude, diversité ou désaccord entre modèles. Le cycle devient collecte $\rightarrow$ sélection $\rightarrow$ annotation $\rightarrow$ entraînement. L’échantillon sélectionné n’est pas représentatif par construction ; le Test doit rester indépendant.

11.5 Augmentations géométriques et photométriques

Les transformations géométriques modifient position et forme : crop, flip, rotation, perspective. Les transformations photométriques modifient intensité, contraste, couleur ou bruit. Leur validité dépend du domaine : un flip peut être acceptable pour un animal et incorrect pour un texte ou une latéralité médicale.

11.6 CutMix et effacement

Cutout masque une région et réduit la dépendance à un détail unique. CutMix insère une région d’une autre image et mélange les labels proportionnellement à l’aire :

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j.$$

Ces méthodes régularisent la classification mais doivent être adaptées avec prudence aux tâches de localisation.

11.7 Contrôle des transformations

Afficher systématiquement des batches augmentés avec annotations superposées. Vérifier coordonnées, masques, interpolation et disparition d’objets. L’interpolation nearest-neighbor est généralement nécessaire pour les masques de classes afin de ne pas créer de labels intermédiaires.

11.8 Versionner les données

Conserver identifiants, source, groupe, date, version du label et split. Un changement silencieux de données empêche de comparer les expériences. Documenter aussi les doublons supprimés et les règles de nettoyage.

Attention. La data augmentation ne corrige pas un biais de collecte. Elle crée des variantes d'exemples existants, mais n'invente pas un capteur, une population ou un contexte totalement absent.

Chapitre 12

Métriques et évaluation

En classification :

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = 2 \frac{PR}{P + R}.$$

Macro moyenne les classes également ; Weighted pondère par support ; Micro agrège les décisions.

12.1 Détection

Une courbe precision-recall est obtenue en faisant varier le seuil. L’Average Precision résume la courbe ; la mAP moyenne sur les classes et, selon le protocole, plusieurs seuils d’IoU. Toujours annoncer le protocole exact, car “mAP” seul est ambigu.

12.2 Segmentation et calibration

Rapporter IoU/Dice par classe et macro, pas seulement la pixel accuracy. Pour la calibration, comparer confiance et fréquence empirique de succès. L’ECE s’écrit

$$ECE = \sum_m \frac{|B_m|}{N} |acc(B_m) - conf(B_m)|.$$

Évaluer aussi latence, débit, mémoire, taille du modèle et consommation lorsque la production l’exige.

12.3 ROC-AUC et PR-AUC

ROC trace TPR contre FPR , avec

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

Avec une classe positive rare, PR-AUC est souvent plus informative car elle expose directement le compromis precision–recall. Une AUC mesure un classement sur tous les seuils ; elle ne choisit pas le seuil opérationnel.

12.4 Balanced Accuracy et MCC

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}.$$

Le coefficient de Matthews binaire est

$$MCC = \frac{TPTN - FPFN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

Ces métriques restent informatives en situation déséquilibrée, mais doivent être accompagnées d’une lecture métier des erreurs.

12.5 Intervalles et répétitions

Une seule seed peut donner une estimation instable. Répéter les expériences importantes, rapporter moyenne et dispersion, et utiliser bootstrap ou validation adaptée pour quantifier l’incertitude. Les images corrélées doivent être rééchantillonnées au niveau du groupe.

12.6 Tests par sous-groupes

Rapporter les performances par classe, site, capteur, qualité et sous-population pertinente. Une moyenne masque les échecs localisés. Les sous-groupes sont définis avant l’analyse lorsque possible afin de limiter la sélection opportuniste.

12.7 Seuil et coût attendu

Avec des coûts C_{FP} et C_{FN} , un critère simple est

$$\text{Cost}(\tau) = C_{FP}FP(\tau) + C_{FN}FN(\tau).$$

Le seuil minimisant ce coût sur Validation peut différer de 0.5. Il faut aussi respecter la capacité opérationnelle, par exemple un nombre maximal d’alertes quotidiennes.

12.8 Protocoles reproductibles

Une comparaison valide garde fixes split, métriques, prétraitement et budget. Rapporter version des données, seed, checkpoint choisi et règle d’arrêt. Le Test n’est évalué qu’après la sélection finale.

Réflexe. Une métrique répond à une question. Avant de la calculer, préciser l'unité d'analyse, la classe importante, le seuil, l'agrégation et le coût des erreurs.

Chapitre 13

Robustesse, explicabilité et déploiement

Tester les changements d'éclairage, capteur, compression, flou, météo, population et domaine. Un modèle peut apprendre un raccourci corrélé au label. L'analyse d'erreurs doit croiser classe, confiance, sous-groupe et condition d'acquisition.

13.1 Explicabilité

Grad-CAM pondère les feature maps par les gradients de la classe :

$$\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial z_c}{\partial A_{ij}^k}, \quad L^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right).$$

Une carte d'activation explique le comportement du modèle, pas une causalité dans le monde réel.

13.2 Production

Exporter avec le prétraitement, l'ordre des classes, les seuils et les versions. Optimisations possibles : quantification, pruning, distillation, compilation et batching. Surveiller dérive des entrées, confiance, taux de rejet, latence et performance lorsque les labels arrivent.

13.3 Tests de robustesse

Construire une matrice de conditions : lumière, flou, compression, résolution, appareil, site et période. Mesurer la dégradation relative

$$\Delta M = M_{condition} - M_{reference}.$$

Les corruptions synthétiques complètent mais ne remplacent pas des données réelles. Tester aussi les entrées hors domaine et les images invalides.

13.4 Incertitude et rejet

Une règle de rejet simple accepte une prédiction seulement si

$$\max_k p_k \geq \tau_r.$$

Évaluer simultanément couverture (proportion acceptée) et risque sur les cas acceptés. Monte-Carlo Dropout, ensembles ou méthodes conformes peuvent enrichir l'estimation, mais aucune méthode ne rend le modèle infallible.

13.5 Compression

La quantification réduit la précision numérique des poids et activations. Le pruning supprime des paramètres ou canaux. La distillation entraîne un élève avec les sorties d'un professeur, par exemple

$$\mathcal{L} = \alpha \mathcal{L}_{labels} + (1 - \alpha) T^2 KL(p_T^{teacher} \| p_T^{student}).$$

Toute optimisation doit être réévaluée sur les métriques et le matériel cible.

13.6 Monitoring

Sans labels immédiats, surveiller distribution des pixels, résolution, taux d'échec, confiance, classes prédites, latence et taux de rejet. Lorsque les labels arrivent, suivre métriques par période et sous-groupe. Définir seuils d'alerte, procédure d'investigation et conditions de réentraînement.

13.7 Sécurité et confidentialité

Contrôler données personnelles, droits d'usage, conservation, accès et traçabilité. Les attaques peuvent inclure exemples adversariaux, empoisonnement de données et extraction du modèle. Le niveau de protection dépend de la menace et de l'impact métier.

13.8 Décision finale d'architecture

La sélection combine qualité, calibration, robustesse, latence, mémoire, coût d'annotation et maintenabilité. Le meilleur score hors ligne n'est pas nécessairement le meilleur système. Documenter les limites connues et prévoir une voie de repli.

13.9 Choisir une approche face à un problème réel

Réflexe. Une classe globale : classification. **Objets et positions** : détection. **Contours précis** : segmentation. **Peu de données** : backbone pré-entraîné. **Temps réel/embarqué** : modèle léger et résolution maîtrisée. **Usage critique** : calibration, rejet, audit par sous-groupes et validation humaine.

13.10 Protocole final

1. Définir la décision métier, les erreurs coûteuses et les contraintes.
2. Choisir la tâche et le niveau minimal d'annotation.
3. Auditer labels, groupes, diversité et représentativité.
4. Construire Train, Validation et Test avant l'augmentation.
5. Créer une baseline simple et un modèle pré-entraîné.
6. Choisir loss et métriques cohérentes avec le problème.
7. Entraîner, suivre les courbes et conserver le meilleur checkpoint.
8. Évaluer par classe, sous-groupe, condition et niveau de confiance.
9. Inspecter visuellement les erreurs et rechercher les raccourcis.
10. Mesurer le coût réel : latence, mémoire, débit et énergie.
11. Déployer avec prétraitement et post-traitement versionnés.
12. Surveiller les dérives et organiser le réentraînement.

13.11 Conclusion

Le meilleur système n'est pas nécessairement le réseau le plus récent. C'est celui qui utilise une formulation correcte, des données représentatives, une évaluation honnête et une architecture compatible avec les contraintes réelles. La progression recommandée est :

```
Baseline simple -> modele pre-entraîne -> fine-tuning  
                -> analyse d'erreurs -> optimisation production
```


Exercices transversaux corrigés

Exercice 1 – Dimensions

Une entrée 128×128 traverse une convolution $K = 3, P = 1, S = 2$. La sortie vaut

$$\left\lfloor \frac{128 + 2 - 3}{2} \right\rfloor + 1 = 64.$$

Avec 32 filtres, sa forme est $32 \times 64 \times 64$.

Exercice 2 – Choix de tâche

On veut mesurer la surface de chaque défaut sur une pièce. Une classification ne localise rien et une détection ne fournit qu'un rectangle. Il faut une segmentation, d'instances si plusieurs défauts doivent être séparés.

Exercice 3 – Déséquilibre

Sur 10 000 images, 100 sont positives. Prédire toujours négatif donne 99% d'accuracy mais un recall positif nul. Il faut examiner recall, precision, PR-AUC et coût des erreurs, puis choisir le seuil sur Validation.

Exercice 4 – IoU

Si $|A| = 100$, $|B| = 80$ et $|A \cap B| = 60$, alors

$$IoU = \frac{60}{100 + 80 - 60} = 0.5.$$

Exercice 5 – Transfer Learning

Pour 1 000 images proches du domaine naturel, commencer par une tête neuve sur backbone gelé. Dégeler ensuite le dernier stage avec faible learning rate si la validation plafonne. Utiliser un split par groupe et le prétraitement des poids.

Exercice 6 – Diagnostic

Train F1 vaut 0.99 et Validation F1 0.62 : suspecter overfitting, fuite, split non représentatif ou raccourci. Inspecter les courbes, groupes, doublons et erreurs avant d'agrandir le modèle.

Exercice 7 – Production

Deux modèles ont mAP 0.51 et 0.49, mais des latences de 180 ms et 25 ms. Pour une contrainte temps réel de 40 ms, le second est le seul directement admissible ; il faut encore vérifier recall des classes critiques et robustesse.

Bibliographie sélective

Bibliographie

- [1] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
- [2] A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, version en ligne.
- [3] A. Krizhevsky, I. Sutskever, G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks”, 2012.
- [4] K. Simonyan, A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition”, 2014.
- [5] K. He et al., “Deep Residual Learning for Image Recognition”, 2015.
- [6] G. Huang et al., “Densely Connected Convolutional Networks”, 2016.
- [7] M. Tan, Q. Le, “EfficientNet : Rethinking Model Scaling for CNNs”, 2019.
- [8] Z. Liu et al., “A ConvNet for the 2020s”, 2022.
- [9] S. Ren et al., “Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks”, 2015.
- [10] J. Redmon et al., “You Only Look Once : Unified, Real-Time Object Detection”, 2016.
- [11] O. Ronneberger et al., “U-Net : Convolutional Networks for Biomedical Image Segmentation”, 2015.
- [12] K. He et al., “Mask R-CNN”, 2017.
- [13] A. Dosovitskiy et al., “An Image is Worth 16x16 Words”, 2020.
- [14] N. Carion et al., “End-to-End Object Detection with Transformers”, 2020.
- [15] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision”, 2021.

Annexe A

Travaux pratiques : classification complète

A.1 Objectif et structure du projet

On souhaite classifier des images rangées par classe. Le projet sépare configuration, données, modèle, entraînement et évaluation. Une structure reproductible est :

```
project/  
  data/{train,val,test}/classe_x/  
  src/{data.py,model.py,train.py,evaluate.py}  
  configs/baseline.yaml  
  checkpoints/
```

Le split doit être créé une fois, versionné et fondé sur les groupes indépendants. Avant tout entraînement, afficher la distribution des classes, les dimensions, quelques images et leurs labels.

A.2 Dataset et transformations

```
from torchvision import datasets, transforms  
  
train_tf = transforms.Compose([  
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),  
    transforms.RandomHorizontalFlip(),  
    transforms.ColorJitter(0.2, 0.2, 0.2, 0.05),  
    transforms.ToTensor(),  
    transforms.Normalize(mean, std),  
)  
eval_tf = transforms.Compose([  
    transforms.Resize(256), transforms.CenterCrop(224),  
    transforms.ToTensor(), transforms.Normalize(mean, std),  
)
```

```
train_ds = datasets.ImageFolder("data/train", train_tf)
val_ds = datasets.ImageFolder("data/val", eval_tf)
assert train_ds.class_to_idx == val_ds.class_to_idx
```

L'augmentation n'est appliquée qu'au Train. Le Test utilise une transformation déterministe identique à celle prévue en production.

A.3 Chargement et reproductibilité

```
import random, numpy as np, torch
from torch.utils.data import DataLoader

seed = 42
random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
train_loader = DataLoader(train_ds, batch_size=32,
                           shuffle=True, num_workers=4, pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=64,
                        shuffle=False, num_workers=4, pin_memory=True)
```

Le shuffle concerne le Train, pas Validation. Fixer une seed améliore la reproductibilité mais ne garantit pas un résultat bit-à-bit sur tout matériel.

A.4 Modèle pré-entraîné

```
from torch import nn
from torchvision.models import resnet18, ResNet18_Weights

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for p in model.parameters(): p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, len(train_ds.classes))
model = model.to(device)
```

La première phase entraîne seulement la tête. La seconde dégèle `layer4` avec un learning rate inférieur.

A.5 Fonctions Train et Validation

```
def run_epoch(model, loader, criterion, optimizer=None):
    training = optimizer is not None
    model.train(training)
    total_loss = total_ok = total_n = 0
```

```

context = torch.enable_grad() if training else torch.no_grad
()
with context:
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        if training: optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)
        if training:
            loss.backward(); optimizer.step()
        total_loss += loss.item() * len(y)
        total_ok += (logits.argmax(1) == y).sum().item()
        total_n += len(y)
    return total_loss / total_n, total_ok / total_n

```

Multiplier la loss moyenne du batch par sa taille évite de biaiser la moyenne lorsque le dernier batch est plus petit.

A.6 Checkpoint et Early Stopping

```

best_loss, patience, stale = float("inf"), 5, 0
for epoch in range(50):
    tr_loss, tr_acc = run_epoch(model, train_loader,
                                criterion, optimizer)
    va_loss, va_acc = run_epoch(model, val_loader, criterion)
    if va_loss < best_loss:
        best_loss, stale = va_loss, 0
        torch.save({"model": model.state_dict(),
                    "classes": train_ds.classes,
                    "epoch": epoch}, "checkpoints/best.pt")
    else:
        stale += 1
        if stale >= patience: break

```

Le meilleur checkpoint est choisi sur Validation. Le Test n'intervient ni dans l'arrêt ni dans le réglage.

A.7 Questions corrigées

Pourquoi ne pas utiliser Softmax dans le modèle ? CrossEntropyLoss combine log-softmax et négative log-vraisemblance de manière stable.

Pourquoi sauvegarder les classes ? L'indice de sortie n'a de sens qu'avec la correspondance indice-nom.

Pourquoi commencer par un backbone gelé ? Cela fournit une baseline rapide et

limite l'overfitting sur petit dataset.

Quand dégeler ? Lorsque la tête a convergé et que l'analyse suggère que les features génériques sont insuffisantes.

Annexe B

Travaux pratiques : détection d'objets

B.1 Audit des annotations

Pour chaque image, vérifier existence du fichier, dimensions positives des boîtes, coordonnées dans l'image, classes autorisées et absence de boîtes minuscules accidentelles. Afficher un échantillon avec boîtes et labels. Calculer nombres d'objets par classe, distribution des aires et ratios.

Une aire normalisée est

$$a = \frac{wh}{HW}.$$

Étudier les quantiles de a révèle si le problème est dominé par de petits objets.

B.2 Transformation cohérente

Pour un redimensionnement de (W, H) vers (W', H') sans padding :

$$x' = x \frac{W'}{W}, \quad y' = y \frac{H'}{H}.$$

Pour un flip horizontal :

$$x'_{min} = W - x_{max}, \quad x'_{max} = W - x_{min}.$$

Après crop, traduire les coordonnées, les borner et supprimer les boîtes dont l'aire visible devient trop faible.

B.3 NMS simplifiée

```
def nms(boxes, scores, threshold):
    order = scores.argsort(descending=True)
    keep = []
    while len(order):
        i = order[0]
        keep.append(i)
        if len(order) == 1: break
        overlap = box_iou(boxes[i:i+1], boxes[order[1:]])
        order = order[1:][overlap <= threshold]
    return torch.stack(keep)
```

En pratique, utiliser l'opération optimisée de la bibliothèque et appliquer NMS séparément par classe lorsque le protocole le demande.

B.4 Analyse d'erreurs

Construire une table contenant classe réelle, classe prédite, confiance, meilleure IoU et taille. Séparer : classification correcte mais localisation insuffisante ; objet correctement localisé mais mauvaise classe ; objet manqué ; doublon ; hallucination sur fond. Une mAP unique ne distingue pas ces causes.

B.5 Étude de résolution

Entraîner ou évaluer à plusieurs résolutions sous budget comparable. Rapporter AP_{small} , latence, mémoire et débit. Si les petits objets gagnent fortement avec la résolution, envisager tiling ou caméra plus proche plutôt qu'un réseau toujours plus profond.

B.6 Exercice corrigé

Deux boîtes ont des aires 120 et 100, intersection 60. Leur union vaut $120 + 100 - 60 = 160$ et

$$IoU = 60/160 = 0.375.$$

À un seuil 0.5, cette association est insuffisante ; à 0.3 elle peut être acceptée, selon le protocole.

Annexe C

Travaux pratiques : segmentation

C.1 Dataset masque-image

Chaque image doit correspondre exactement à son masque. Les transformations aléatoires partagent les mêmes paramètres géométriques. Les images utilisent interpolation bilinéaire ; les masques de classes utilisent nearest-neighbor.

```
image = resize(image, size, interpolation=BILINEAR)
mask = resize(mask, size, interpolation=NEAREST)
assert set(unique(mask)).issubset(valid_class_ids)
```

C.2 Loss combinée

Pour un binaire déséquilibré :

$$\mathcal{L} = \lambda \mathcal{L}_{BCE} + (1 - \lambda) \mathcal{L}_{Dice}.$$

Cette combinaison fournit un signal pixel par pixel et un signal de recouvrement global. Le choix de λ est validé ; il n'existe pas de valeur universelle.

C.3 Évaluation par classe et par image

Accumuler une matrice de confusion pixel par pixel permet de calculer IoU par classe. Rapporter aussi médiane et quantiles de Dice par image : une moyenne élevée peut masquer des images totalement ratées. Pour les images sans objet, définir explicitement le traitement du cas où prédiction et vérité sont toutes deux vides.

C.4 Tiling avec recouvrement

Une grande image est découpée en fenêtres. Les probabilités dans les zones de chevauchement sont moyennées avec une pondération plus forte au centre afin de réduire les coutures. Le split est fait avant le tiling au niveau du patient ou de l'image source.

C.5 Diagnostic visuel

Produire pour chaque cas : image, vérité, masque prédit, probabilité et carte d'erreur. Les faux positifs et faux négatifs peuvent être colorés différemment. Trier par Dice révèle rapidement annotations incohérentes, objets minuscules et changements de domaine.

Annexe D

Études de cas guidées

D.1 Défauts industriels

Besoin : localiser des défauts rares, parfois minuscules.

Tâche : segmentation ou détection selon le besoin de surface.

Split : par lot, machine et période.

Risques : fond corrélé, changement d'éclairage, très peu de positifs.

Protocole : baseline de traitement d'image, backbone pré-entraîné, crops haute résolution, recall à un taux de fausses alertes admissible.

Production : seuil de rejet, revue humaine et collecte continue des erreurs.

D.2 Imagerie médicale

Le split est réalisé par patient. Il faut documenter appareil, site, protocole et population. Une augmentation anatomiquement invalide est interdite. Rapporter sensibilité, spécificité, calibration et intervalles d'incertitude ; comparer les sous-groupes et utiliser une validation externe. Le modèle assiste une décision et ne remplace pas automatiquement l'expertise clinique.

D.3 Caméra embarquée

Le système détecte objets et piétons avec contrainte de latence. Évaluer de bout en bout au batch 1, y compris décodage et NMS. Tester nuit, pluie, flou de mouvement et objets éloignés. Quantification et modèle léger sont envisagés seulement après mesure de leur impact sur recall critique.

D.4 Catalogue produit

Un classifieur doit reconnaître de nombreuses catégories proches. Utiliser métriques top-1/top-k, Macro F1 et analyse hiérarchique. Les photographies d'un même produit restent dans le même split. Un modèle vision-langage peut fournir une baseline zero-shot, puis être comparé à un fine-tuning supervisé.

Annexe E

Exercices supplémentaires corrigés

E.1 Réseaux et optimisation

Question : pourquoi une succession de couches linéaires sans activation reste-t-elle linéaire ?

Correction : $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, soit une unique transformation affine.

Question : une loss Train ne diminue jamais. Quel ordre de diagnostic ?

Correction : visualiser données/labels, contrôler formes et types, vérifier logits/loss, gradients et optimiseur, puis learning rate. Tester enfin l'overfit d'un petit batch.

E.2 Classification

Une classe rare obtient precision 0.8 et recall 0.2. Le modèle émet peu de faux positifs mais manque 80% des vrais cas. Si les FN sont critiques, abaisser le seuil ou modifier entraînement/données, puis mesurer le nouveau compromis sur Validation.

E.3 Détection

Un modèle conserve trois boîtes sur le même objet. C'est un problème de doublons. Vérifier seuil NMS, calibration des scores et apprentissage. Augmenter le seuil NMS conserve généralement davantage de boîtes ; le diminuer supprime plus agressivement, avec risque pour les objets proches.

E.4 Segmentation

La pixel accuracy vaut 99% et Dice objet 0. Le fond occupe 99% des pixels et le modèle prédit toujours fond. Employer IoU/Dice par classe, pondération ou loss adaptée, et

améliorer la présence des objets dans les batches.

E.5 Transformers

Avec une image 224×224 et des patches 16×16 , le nombre de tokens image est

$$(224/16)^2 = 196.$$

Diviser la taille des patches par deux multiplie le nombre de tokens par quatre et le coût d'attention globale approximativement par seize.

E.6 Question de synthèse

Proposer un pipeline pour 2 000 images de cinq classes avec plusieurs vues par objet.

Correction : split par objet, audit des classes, transformations déterministes pour Validation/Test, backbone pré-entraîné gelé, tête cinq logits et Cross-Entropy, augmentation plausible, puis fine-tuning progressif. Évaluer matrice de confusion, Macro F1, calibration et erreurs par condition.

Annexe F

Glossaire

- Backbone** Réseau extrayant les représentations visuelles.
- Batch** Groupe d'exemples traité lors d'une itération.
- Bounding box** Rectangle localisant un objet.
- Calibration** Accord entre confiance annoncée et fréquence réelle de succès.
- Checkpoint** État sauvegardé du modèle et de l'entraînement.
- Data augmentation** Transformation plausible utilisée pour enrichir le Train.
- Domain shift** Différence de distribution entre entraînement et usage.
- Epoch** Passage complet sur le dataset d'entraînement.
- Feature map** Carte produite par un filtre ou un bloc.
- Fine-tuning** Adaptation de poids pré-entraînés à une nouvelle tâche.
- IoU** Rapport intersection sur union de deux régions.
- Kernel** Petit tenseur de poids appliqué localement.
- Logit** Score brut avant sigmoid ou softmax.
- mAP** Moyenne d'Average Precision selon un protocole donné.
- Neck** Module fusionnant des features, souvent multi-échelles.
- NMS** Suppression de détections redondantes.
- Padding** Valeurs ajoutées autour d'une entrée.
- Patch** Région d'image traitée comme token ou sous-image.
- Receptive field** Région d'entrée influençant une activation.
- Stride** Pas de déplacement d'un kernel.
- Transfer Learning** Réutilisation de connaissances pré-entraînées.

Annexe G

Aide-mémoire mathématique

Annexe H

Fiches de révision par chapitre

H.1 Fiche 1 – Formuler le problème

Questions : sortie globale, objets, pixels, points ou texte ? Quel coût pour FP et FN ?
Quelle unité de split ?

Tâches : classification, multilabel, détection, segmentation sémantique/d’instances, pose, profondeur.

Pièges : groupes corrélés entre Train et Test ; contexte absent ; annotations ambiguës.

Réflexe : choisir la sortie minimale utile et un Test représentatif de la production.

H.2 Fiche 2 – Convolution

Formule : $N_{out} = \lfloor (N + 2P - D(K - 1) - 1) / S \rfloor + 1$.

Paramètres : $K_h K_w C_{in} C_{out} + C_{out}$.

Notions : kernel, feature map, stride, padding, dilation, gradient, Sobel.

Piège : convolution des frameworks = souvent corrélation croisée ; l'équivalence est sans conséquence lorsque les poids sont appris.

H.3 Fiche 3 – Apprentissage neuronal

Neurone : $z = w^T x + b$, $a = f(z)$.

Sorties : sigmoid pour binaire/multilabel ; softmax pour multiclasse.

Mise à jour : $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$.

Diagnostic : données, formes, logits/loss, gradients, optimiseur, petit batch, puis courbes Train/Validation.

H.4 Fiche 4 – CNN

Principes : connectivité locale, partage des poids, hiérarchie et champ réceptif.

Bloc : Conv $\rightarrow$ Norm $\rightarrow$ activation $\rightarrow$ réduction.

Tendance : résolution $\downarrow$, canaux et sémantique $\uparrow$.

Pièges : ordre NCHW/NHWC, mauvais Flatten, softmax en double, oubli de `eval()`.

H.5 Fiche 5 – Architectures et transfert

VGG : petits kernels. **Inception** : multi-échelle. **ResNet** : $y = F(x) + x$. **MobileNet** : depthwise separable.

Transfert : nouvelle tête, backbone gelé, puis dégel progressif.

Règle : $\eta_{early} < \eta_{late} < \eta_{head}$.

Piège : poids corrects avec mauvais prétraitement.

H.6 Fiche 6 – Classification

Binaire : un logit + BCE. **Multiclasse** : K logits + CE. **Multilabel** : K sigmoids + BCE.

Évaluation : confusion, precision, recall, Macro F1, calibration et sous-groupes.

Déséquilibre : pondération, sampling, focal loss, collecte.

Piège : accuracy élevée grâce à la classe majoritaire ou à un raccourci visuel.

H.7 Fiche 7 – Détection

Sortie : classe + confiance + boîte.

IoU : $|A \cap B| / |A \cup B|$.

Évaluation : appariement unique, PR, AP, mAP et résultats par taille.

Pièges : conventions de coordonnées, boîtes non transformées, doublons, petits objets perdus.

H.8 Fiche 8 – YOLO

Structure : backbone $\rightarrow$ neck $\rightarrow$ head.

Loss : boîte + objectness + classe.

Inférence : letterbox, réseau, décodage, seuil, NMS, retour aux coordonnées.

Choix : résolution contre latence ; anchor-based contre anchor-free ; mesurer la chaîne complète.

H.9 Fiche 9 – Segmentation

Sémantique : classe par pixel. **Instance** : masque par objet. **Panoptique** : things + stuff.

U-Net : encodeur, décodeur et skip connections.

Métriques : $Dice = 2|P \cap G|/(|P| + |G|)$ et IoU.

Pièges : pixel accuracy dominée par le fond ; interpolation non-nearest des masques.

H.10 Fiche 10 – Vision Transformers

Tokens : $N = HW/P^2$.

Attention : $\text{softmax}(QK^T/\sqrt{d})V$.

Modèles : ViT, Swin, DETR, CLIP.

Pièges : coût quadratique, besoin de pré-entraînement, hallucinations multimodales et faiblesse sur certains détails spatiaux.

H.11 Fiche 11 – Données et augmentation

Taxonomie : définition, exclusions, cas limites et ignore.

Augmentations : géométriques, photométriques, MixUp, CutMix.

Règle : $y(T(x)) = y(x)$ pour une transformation censée conserver le label.

Piège : l'augmentation ne remplace pas une population, un capteur ou un contexte absent.

H.12 Fiche 12 – Évaluation

Classification : F1, ROC-AUC, PR-AUC, MCC.

Détection : préciser seuils IoU et protocole mAP.

Segmentation : IoU/Dice par classe et par image.

Toujours ajouter : incertitude, sous-groupes, latence, mémoire et coût métier.

H.13 Fiche 13 – Robustesse et production

Tester : lumière, capteur, compression, flou, domaine et hors-distribution.

Optimiser : quantification, pruning, distillation, batching.

Surveiller : dérive, confiance, classes, rejet, latence et performance retardée.

Réflexe : versionner prétraitement, poids, classes, seuils et post-traitement ; prévoir une voie de repli.

Annexe I

Aide-mémoire mathématique

$$\begin{aligned}\sigma(z) &= \frac{1}{1 + e^{-z}}, & p_k &= \frac{e^{z_k}}{\sum_j e^{z_j}} \\ N_{out} &= \left\lfloor \frac{N + 2P - D(K - 1) - 1}{S} \right\rfloor + 1 \\ IoU &= \frac{|A \cap B|}{|A \cup B|}, & Dice &= \frac{2|A \cap B|}{|A| + |B|} \\ \theta_{t+1} &= \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)\end{aligned}$$